

Opcenter RAG - Hiring Manager Interview Questions

60 realistic project questions with strong answer structures, likely follow-ups, honest status language, and a last-minute rehearsal plan.

PROJECT	Opcenter manual-grounded RAG assistant
FOCUS	Architecture, decisions, evaluation, LLMOps and interview delivery
REPOSITORY	github.com/nagateja2004/op-center-bot

Contents

Do not memorize every sentence. Learn the structure, then answer in your own words.

Contents	2
How to use this question bank	3
STAR template for project questions	3
A. Product and architecture	4
B. Ingestion, retrieval and context	7
C. LangGraph, LLMs, memory and safety	9
D. API, scaling, security and operations	11
E. Evaluation, LLMOps and hiring-manager depth	13
F. Ownership, trade-offs and behavioral questions	15
Last-minute rapid revision	17
Architecture in one breath	17
Evaluation in one breath	17
Current status in one breath	17
Numbers worth remembering	17
Interview-day checklist	18
Questions you should ask the interviewer	18

How to use this question bank

- Round 1: answer each question aloud in 30 to 60 seconds without reading the model answer.
- Round 2: compare your answer and add one missing reason, trade-off or measured fact.
- Round 3: practice the likely follow-up. This is where interviewers test real ownership.
- Use 'I implemented' only for repository code. Use 'I configured and verified' only where the environment actually ran.
- If you do not know a detail, explain how you would measure or verify it instead of guessing.

Three answers to master first

One-minute overview; complete request flow; how evaluation works. If these three are clear, most follow-ups become easier.

STAR template for project questions

Part	What to say
Situation	The manual-search or production-quality problem.
Task	The property you needed: grounding, recall, latency, scale or regression safety.
Action	The concrete design and why you selected it.
Result	A verified artifact, test, metric or honest remaining gap.

A. Product and architecture

Hiring managers use these questions to test whether you understand the problem and can explain the system without hiding behind framework names.

01

Give me a one-minute overview of the project.

Strong answer	I built an asynchronous RAG assistant grounded in Opcenter PDF manuals. The offline pipeline turns PDFs into structure-aware EvidenceUnits, searchable segments, Chroma vectors and a BM25 index. Online, a LangGraph workflow understands the question, performs hybrid retrieval, restores context, reranks, grades evidence, generates a cited answer and verifies it. FastAPI streams results to Streamlit, PostgreSQL stores thread checkpoints, Redis coordinates two replicas, and LangSmith provides traces. I also added a 50-case golden dataset, deterministic and semantic evaluation, and CI gate automation.
Likely follow-up	Follow-up: Which parts are fully validated? Say that the code and contract checks are implemented, while the full live 50-case release gate and automatic server deployment still require staging configuration.

02

What problem were you solving?

Strong answer	Large technical manuals are difficult to navigate, and ordinary keyword search misses paraphrases. A general LLM can give a fluent but unsupported answer. The project aims to return fast, source-grounded answers for definitions, procedures, comparisons and troubleshooting while refusing unsupported questions. The engineering goal was traceability: every important answer should map back to retrieved manual evidence and be diagnosable when it fails.
Likely follow-up	Follow-up: Define success using grounding, citation validity, routing accuracy, latency and safe fallback behavior.

03

Why did you use RAG instead of fine-tuning?

Strong answer	The knowledge is document-specific and changes when manuals change. RAG lets me update knowledge by re-ingesting documents, preserves direct source citations and allows the system to say when evidence is missing. Fine-tuning would be less transparent for factual updates and would not automatically provide the exact supporting page. Fine-tuning could still help style or intent classification, but it would not replace retrieval for this use case.
Likely follow-up	Follow-up: Mention that RAG shifts risk toward ingestion and retrieval quality, so those layers need explicit evaluation.

04

Who is the intended user?

Strong answer	The primary users are manufacturing engineers, Opcenter consultants and support engineers who need definitions, setup steps, field meanings or relationships from manuals. The current UI is anonymous and session-based, so it is a technical assistant rather than a user-account platform. If deployed across teams, I would add authentication, document entitlements and retention controls.
Likely follow-up	Follow-up: Explain why anonymous UUID isolation is not the same as enterprise authentication.

05 Walk me through the high-level architecture.

Strong answer	Streamlit sends a request to FastAPI. FastAPI stores request state in Redis and runs a compiled LangGraph workflow using PostgreSQL checkpoints. The graph calls hybrid retrieval backed by Chroma and BM25, deterministic context processing, a local cross-encoder and role-specific Groq models. LangSmith traces the graph, while the evaluator sends golden questions through the same API and feeds deterministic and semantic results into a CI gate. Docker Compose packages the frontend, two backend replicas, PostgreSQL, Redis and Chroma.
Likely follow-up	Follow-up: Draw three planes - online serving, offline knowledge preparation, and quality/operations.

06 Why separate Streamlit from FastAPI?

Strong answer	It keeps the interface independent from the RAG service. FastAPI owns validation, request lifecycle, SSE, health checks and orchestration; Streamlit only manages browser state and rendering. That separation makes the backend testable without a UI, allows a different frontend later, and supports multiple backend replicas. It also prevents the browser from directly handling model keys or database connections.
Likely follow-up	Follow-up: Explain that only the frontend is exposed in the default Compose setup; the backend remains on the internal network.

07 Why use LangGraph instead of one long chain?

Strong answer	The workflow has real decisions: basic chat can exit early, insufficient evidence can retry once, unsupported questions use a fallback, and diagrams run only for grounded answers. LangGraph makes those branches explicit, keeps a typed state object, supports checkpointing and produces node-level traces. A simple chain would hide control flow in prompt code and make bounded retry and state recovery harder to reason about.
Likely follow-up	Follow-up: Name the exact nodes and the conditions after understanding, grading and verification.

08 What are the major stateful components?

Strong answer	PostgreSQL stores LangGraph conversation checkpoints. Redis stores pending requests, thread ownership, caches, request status and distributed model admission limits. Chroma stores dense vector indexes, while BM25 and EvidenceUnit metadata are versioned local index artifacts. LangSmith stores optional observability traces, not application conversation memory.
Likely follow-up	Follow-up: Contrast durable conversation state with operational coordination state.

09 What happens if a backend replica fails?

Strong answer	The request and ownership records are in Redis rather than only in process memory, and the conversation checkpoint is in PostgreSQL. Another healthy replica can handle later requests because the important shared state is external. An in-flight streaming request may still fail and the client must retry, but the architecture avoids tying every conversation to one process. Readiness checks remove replicas whose dependencies or models are unavailable.
Likely follow-up	Follow-up: Mention that exactly-once answer delivery is not claimed; the request store uses atomic pop and bounded TTLs.

10

What is the most important architectural boundary?

Strong answer	Knowledge preparation is offline. Manual parsing, EvidenceUnit creation and index rebuilding never happen during startup or a chat request. That keeps latency predictable, prevents two replicas from rebuilding shared state and makes the deployed index version auditable. Online serving only loads and queries already validated artifacts.
Likely follow-up	Follow-up: State when ingestion should rerun - manual changes or index schema changes.

B. Ingestion, retrieval and context

These questions test whether you understand RAG quality below the prompt layer.

11 How are the PDFs ingested?

Strong answer	PyMuPDF extracts embedded text, headings, tables, procedures, warnings and usable figures. The pipeline groups source material into immutable EvidenceUnits, produces smaller RetrievalSegments and search representations, computes embeddings for Chroma, builds a BM25 index and writes heading, concept, figure, manifest and audit artifacts. It validates that JSON, BM25 and Chroma IDs agree. Scanned pages require OCR before this pipeline because embedded text is necessary.
Likely follow-up	Follow-up: Explain why ingestion is an explicit Docker tools profile rather than an application startup step.

12 What is an EvidenceUnit?

Strong answer	An EvidenceUnit is the complete source object used for reasoning and citation. It can represent a coherent text section, procedure or table with stable evidence ID and manual/page metadata. Smaller segments can be retrieved, but the system resolves them back to the parent EvidenceUnit before generation. This avoids answering from a fragment that lost a condition, header or adjacent step.
Likely follow-up	Follow-up: The key phrase is 'search small, answer from complete evidence.'

13 What is a RetrievalSegment?

Strong answer	It is a bounded searchable portion of an EvidenceUnit designed for embeddings and lexical ranking. Segments have their own IDs, word/token counts, previous/next links and a reference to the parent evidence ID. They improve recall and comply with embedding limits while preserving a route back to the full source unit.
Likely follow-up	Follow-up: Explain that segment IDs and evidence IDs are validated across all indexes.

14 Why did you combine Chroma and BM25?

Strong answer	Chroma dense retrieval handles semantic similarity and paraphrases. BM25 handles exact acronyms, class names, fields and product terminology. Technical documentation needs both: vector-only search can blur rare terms, while lexical-only search can miss indirect wording. The combination improves recall across direct and indirect questions.
Likely follow-up	Follow-up: Give the example 'manufacturing equipment' retrieving the manual concept Resource.

15 How do you combine vector and BM25 scores?

Strong answer	I use weighted reciprocal-rank fusion. RRF converts each result's rank position into a contribution, so I do not need to pretend that Chroma similarity and BM25 scores share one calibrated scale. Results that rank highly in either source get credit, and agreement across sources raises confidence. The project uses an RRF constant of 60 and also supports fusion across multiple queries/aspects.
Likely follow-up	Follow-up: Be ready to write the conceptual formula weight divided by k plus rank.

16

Why add a cross-encoder after retrieval?

Strong answer	Embedding retrieval encodes the query and documents separately, which is fast for broad search. A cross-encoder evaluates each query-document pair together and usually provides better final relevance ordering. Because it is more expensive, it only reranks a bounded candidate set locally. If it fails to load or predict, the system retains the RRF order rather than failing the request.
Likely follow-up	Follow-up: Distinguish recall-oriented candidate generation from precision-oriented reranking.

17

What is neighbor expansion?

Strong answer	Strong retrieval hits can begin or end in the middle of a logical explanation. The system follows previous and next segment IDs to add nearby material, labels the relation and deduplicates candidates. It then resolves the selected segments to full EvidenceUnits. This protects procedure order, conditions and local context without retrieving an entire manual page blindly.
Likely follow-up	Follow-up: Explain why expansion happens before final reranking and grading.

18

How does context compression work?

Strong answer	After resolving a complete EvidenceUnit, deterministic code builds an aspect-specific view. For prose it selects lexically relevant sentences plus useful adjacent conditions; for procedures it selects ordered steps or the full procedure when required; for tables it selects relevant rows or all rows when requested. It stores selected indexes and character counts while preserving the original unit unchanged.
Likely follow-up	Follow-up: This is extraction by rules, not abstractive summarization.

19

Did you use LLMChainExtractor or LLMChainFilter?

Strong answer	No. The implemented compressor is deterministic and uses no LLM. That keeps it repeatable, cheap and source-preserving, which is valuable for technical evidence and CI tests. An LLM extractor might improve semantic selection, but it adds latency, cost and another opportunity to omit or rewrite a critical condition. I would only add it after an A/B evaluation proves the gain.
Likely follow-up	Follow-up: Compression occurs after EvidenceUnit resolution and before evidence grading/generation.

20

How do you handle multi-part questions?

Strong answer	The understanding stage extracts required aspects and preserves explicitly named concepts. Retrieval can run aspect-specific queries, and evidence is graded independently for every required aspect. The answer is produced only after the coverage state is sufficient or partial; otherwise one targeted broadening query tries to recover missing evidence. This prevents one well-supported half of a comparison from hiding an unsupported half.
Likely follow-up	Follow-up: Explain why overall top-k relevance is not enough for comparison questions.

C. LangGraph, LLMs, memory and safety

These questions test orchestration, model economics and hallucination controls.

21 Name the LangGraph nodes in order.

Strong answer	The main path is <code>understand_question</code> , <code>retrieve_documents</code> , <code>expand_context</code> , <code>rerank_documents</code> , <code>grade_evidence</code> , <code>generate_answer</code> and <code>verify_answer</code> , followed by <code>generate_diagram</code> only when useful and grounded. The retry path is <code>broaden_query</code> back to <code>retrieve_documents</code> . The refusal path is <code>generate_fallback</code> . Basic chat exits immediately after <code>understand_question</code> .
Likely follow-up	Follow-up: State that <code>retry_count</code> limits broadening to one attempt.

22 How does evidence grading work?

Strong answer	The grader receives bounded evidence for each requested aspect and decides whether support is sufficient, partial, missing or outside scope. Grading per aspect separates retrieval confidence from answer fluency. If evidence is missing on the first pass, the graph can broaden once; otherwise it uses a safe fallback. This makes the system answer only after an explicit support decision.
Likely follow-up	Follow-up: The grader is an LLM call; deterministic citation validation still runs separately later.

23 Why limit retrieval retry to one?

Strong answer	Unlimited agentic retry can create latency spikes, token cost and loops without improving evidence. One targeted broadening attempt captures the common case where the initial query wording was weak while keeping worst-case behavior predictable. After that, the system prefers an honest limitation. Bounded failure behavior is more important than appearing endlessly autonomous.
Likely follow-up	Follow-up: Connect the bound to p95 latency and provider rate limits.

24 How do you prevent hallucinations?

Strong answer	The answer prompt allows only supplied EvidenceUnits, evidence is graded before generation, and citations are required. Deterministic validation checks source IDs, and the verifier corrects the draft using cited evidence only. Unsupported or out-of-scope cases route to a safe deterministic fallback. This reduces hallucination risk, although no LLM system can promise zero hallucinations, so evaluation and monitoring remain necessary.
Likely follow-up	Follow-up: Do not say the verifier alone guarantees truth; it is another model and needs evidence plus deterministic checks.

25 How many LLM calls are made for one question?

Strong answer	The number is dynamic. A typical direct single-aspect question uses three calls: evidence grader, answer model and verifier. Ambiguous or multi-aspect questions may add one planner call; missing evidence may add broadening and another grading pass; a useful diagram adds one. Basic greetings use zero RAG LLM calls, and retrieval, compression, reranking and citation parsing are non-LLM operations.
Likely follow-up	Follow-up: The evaluation judge is outside the user request and adds one call per evaluated case, with one error retry.

26

Why use role-specific models?

Strong answer	Planning, grading, answering and diagram generation have different capability and token needs. Smaller models can handle structured routing and grading, while the larger answer model has more output capacity. The verifier uses a separate policy, which reduces correlated mistakes. Role-specific configuration also lets me tune timeout, temperature, output budget and fallback independently.
Likely follow-up	Follow-up: Mention that this increases operational configuration and must be observed per role/model.

27

How are model failures handled?

Strong answer	Each role has one primary and at most one distinct fallback model. Provider errors are classified and sanitized, retries are bounded, timeouts are role-specific, and Redis admission control limits concurrency, requests per minute, tokens per minute, queue depth and wait time. Diagram failure is non-fatal to the grounded text answer. If essential answer generation cannot complete, the client receives a safe error rather than internal details.
Likely follow-up	Follow-up: Explain why the provider client's automatic retries are disabled in favor of explicit policy.

28

What kind of memory does the bot use?

Strong answer	It uses durable short-term conversation memory through LangGraph checkpoints in PostgreSQL. A thread can use earlier turns to resolve a follow-up, even across backend replicas. It does not create a long-term personal profile or recall unrelated conversations. Starting a new conversation generates a new thread and therefore a new context boundary.
Likely follow-up	Follow-up: LangSmith traces are observability records, not the conversational memory used by the graph.

29

How does a follow-up question work?

Strong answer	The browser sends the same session, conversation and thread UUIDs. Redis confirms that the session owns the thread, and LangGraph loads the previous checkpoint from PostgreSQL. The understanding stage uses the recent messages to resolve a question such as 'How is it related to resources?' into a standalone retrieval intent. The new state is checkpointed after the run.
Likely follow-up	Follow-up: A follow-up-dependent answer is not stored in the final-answer cache because it depends on conversation context.

30

How are citations validated?

Strong answer	Returned sources receive stable [S1], [S2] style IDs corresponding to the selected evidence. The answer is parsed for citation IDs, and the evaluator checks that cited IDs exactly match returned source IDs for sufficient cases. Verification can normalize citation formatting and remove unsupported statements. Stable evidence metadata includes manual, chapter, section and page information for traceability.
Likely follow-up	Follow-up: Explain the difference between citation presence and citation correctness.

D. API, scaling, security and operations

These questions separate a demo from a production-oriented design.

31 Why does the API return HTTP 202 first?

Strong answer	A RAG request can take much longer than ordinary HTTP work. POST /v1/chat validates and accepts the request, stores it in Redis and immediately returns a request ID plus conversation identifiers. The client then opens a separate SSE stream. This avoids holding request creation open, gives progress visibility and creates a clean request lifecycle.
Likely follow-up	Follow-up: Name the progress, answer, complete and error event types.

32 Why use server-sent events instead of WebSockets?

Strong answer	The dominant communication direction is server to browser after one submitted question, so SSE is simpler than a bidirectional socket. It works over normal HTTP, supports event types and is easy to consume from the Streamlit frontend. WebSockets would be more appropriate for continuous two-way interaction, collaborative state or client-to-server streaming. SSE still needs reconnection and proxy timeout consideration.
Likely follow-up	Follow-up: Be honest that the current answer chunks are application-level streamed events, not necessarily raw token streaming from every model.

33 How can two backend replicas share requests?

Strong answer	Redis is the shared handoff layer. The POST endpoint stores a pending request by request ID, and the SSE endpoint atomically pops it, so either replica can continue. Redis also stores thread ownership, status, cache and admission counters. Without this, a load balancer could route POST and GET to different processes and lose the request.
Likely follow-up	Follow-up: PostgreSQL separately stores conversation checkpoints; Redis is not the only state service.

34 How do you handle overload?

Strong answer	The system bounds local inference concurrency and queue depth, and separately enforces per-model Groq concurrency, request and token rates, queue depth and maximum wait. Request records and ownership have TTLs, message/request sizes are limited, and provider timeouts are explicit. When capacity is unavailable, the system fails safely instead of creating an unbounded queue that destroys tail latency.
Likely follow-up	Follow-up: Connect overload protection to 429/503 behavior and p95 monitoring.

35 What do /health, /ready and /metrics mean?

Strong answer	/health is liveness: the process is running. /ready is dependency readiness: graph, PostgreSQL, Redis, Chroma, BM25, EvidenceUnits, embedding model and reranker are usable. /metrics exposes process-local counters, gauges and summaries for HTTP, graph nodes, model calls, tokens and queues. A load balancer should use readiness, not only liveness.
Likely follow-up	Follow-up: The project does not require a Prometheus server; it only exposes scrape-compatible text.

36

What security controls are present?

Strong answer

Secrets and manuals are excluded from source control, evaluation context requires a constant-time token check, and that context is bounded. UUID threads are bound to the originating anonymous session, inputs and request bytes are limited, CORS is configurable, TTLs limit stale records, and errors are sanitized. For enterprise production I would add authenticated users, authorization by document set, secret-manager integration, TLS termination, audit logs and retention/deletion policy.

Likely follow-up

Follow-up: Do not describe UUID possession as strong authentication.

37

How is the project containerized?

Strong answer

Docker Compose defines Streamlit, two FastAPI replicas, PostgreSQL, Redis and Chroma, plus explicit tools profiles for ingestion, evaluation and load testing. PostgreSQL and Chroma use persistent volumes, while manuals and index metadata are bind-mounted and not copied into images. Separate Compose project names and ports are designed for staging and production.

Likely follow-up

Follow-up: Explain why deployment deliberately does not copy .env, manuals or indexes.

38

What does automatic deployment mean here?

Strong answer

GitHub Actions connects to staging or production through SSH, checks out the exact commit in the configured server directory and runs the Docker Compose deployment script. A PR commit should be deployed to staging, evaluated and gated before merge. After merge, the same merged commit should pass the full staging gate before production release. This workflow is implemented but cannot run until the GitHub environments and server secrets are configured.

Likely follow-up

Follow-up: A git push updates GitHub; it does not update the server by itself.

39

What happens if someone forks the repository?

Strong answer

They can manually deploy it with their own .env, manuals, indexes and API keys. GitHub does not share the original repository's secrets with the fork, so automatic SSH deployment will not work until the fork owner configures their own environments and credentials. The delivery workflow also intentionally avoids deploying pull requests from forks because secrets are unavailable and untrusted code must not access them.

Likely follow-up

Follow-up: Mention copyright/data policy for manuals and why they should not be committed.

40

How would you scale to ten times the traffic?

Strong answer

First I would use LangSmith and load-test data to identify whether latency comes from Groq, embeddings, reranking, Redis or database checkpoints. I would add backend replicas behind a load balancer because shared state already lives in Redis/PostgreSQL, set production-grade connection pools, and size per-model admission limits. I would cache only context-independent answers, batch or optimize embeddings/reranking, and aggregate metrics centrally. I would load test p95 and error rate before increasing concurrency blindly.

Likely follow-up

Follow-up: Scaling replicas does not remove external model rate limits; admission control must remain global.

E. Evaluation, LLMOps and hiring-manager depth

These questions test whether you can prove quality, discuss limitations and take ownership.

41 How did you evaluate the application?

Strong answer	I versioned 50 golden questions across ten categories: direct, indirect, follow-up, procedure, table/field, comparison, unsupported, irrelevant and two product-specific sets. Each run stores the question, expected constraints, actual answer, sources, evidence status, latency and optional judge result. Deterministic code measures term, manual, status, citation and diagram accuracy, while a separate LLM judge scores semantic quality. JSON and HTML reports feed fixed thresholds and optional accepted-baseline regression checks.
Likely follow-up	Follow-up: Be explicit that the current stored live result is only a three-case smoke run and the full 50-case live baseline is pending.

42 What is a deterministic evaluator?

Strong answer	It is normal code that applies fixed rules to the response. It checks expected or required terms, expected manuals, evidence status, returned/cited source ID consistency, diagram expectation, errors and latency. The same response produces the same result with no judge tokens. Its weakness is brittleness when a semantically correct answer uses wording outside the expected terms.
Likely follow-up	Follow-up: Deterministic does not mean the RAG output itself is deterministic; only the scoring rule is fixed.

43 What does LLM-as-a-judge add?

Strong answer	It evaluates meaning rather than only exact strings. The judge sees the question, optional conversation context, gold constraints, actual answer, evidence status and protected retrieved evidence. It scores relevance, correctness, faithfulness, completeness and clarity from one to five. This can recognize valid paraphrases, but it introduces model cost and variance, so it is combined with deterministic rules and should be calibrated with human labels.
Likely follow-up	Follow-up: The judge runs at temperature zero with JSON-only output and one invalid-response retry.

44 What is semantic pass rate?

Strong answer	It is the fraction of evaluated cases whose semantic judgment passes the case-level rule. In this project, a case passes when its mean score is at least 4.0 and correctness and faithfulness are each at least 4.0. If 9 of 10 pass, semantic pass rate is 90%. It complements the average because several failures can be hidden by very high scores on other cases.
Likely follow-up	Follow-up: The aggregate default requires at least 85% semantic pass rate and 4.0 average.

45 What is p95 latency?

Strong answer	Sort all request durations; p95 is the value at or below which roughly 95% of requests finish. It shows tail performance, whereas mean or median can hide a small number of very slow requests. The evaluation gate currently uses a 75-second maximum, and the load test also reports p50/p95 time to first answer token. Thresholds should be calibrated to the real service objective.
Likely follow-up	Follow-up: The latest three-case smoke run had 130.5-second p95, so latency optimization remains real work.

46

What exactly is the CI quality gate?

Strong answer	The evaluator exits non-zero if metrics fall below fixed thresholds, exceed error/latency limits or regress more than the allowed amount from an accepted baseline. GitHub Actions treats that non-zero result as a failed check, preventing promotion when branch protection is configured. The gate stores JSON and HTML first so the developer can diagnose the failed cases. It is a release decision mechanism, not an evaluator by itself.
Likely follow-up	Follow-up: Name a few thresholds: 85% overall, 90% manual/status, 95% citations, 4.0 semantic average and zero errors.

47

Why use 10 cases on a PR and 50 after merge?

Strong answer	Live LLM evaluation is slower, costs tokens and can vary slightly. The PR subset selects one representative case from each category for faster feedback. The merged commit runs the full dataset on staging for release confidence and baseline comparison. The subset is not compared with a 50-case baseline because the populations differ.
Likely follow-up	Follow-up: Unit tests and deterministic contracts still run on every relevant change even when the live backend evaluation is skipped.

48

How does LangSmith help you diagnose a failed case?

Strong answer	The HTML report tells me which expected constraint or semantic dimension failed. I then open that case's root LangSmith trace and compare the understanding output, retrieved candidates, expanded EvidenceUnits, reranker order, grader decision, prompt inputs, model output and verifier changes. Node durations show where latency occurred, and model spans show retries/tokens. This lets me classify the problem as ingestion, retrieval, compression, grading, prompting, provider or evaluation-data error.
Likely follow-up	Follow-up: Fix the earliest incorrect stage rather than patching only the final prompt.

49

What is the current validation status?

Strong answer	The repository is merged to main, the evaluation contract passed, and the dataset/evaluator/tracing/workflow code exists. The latest stored live run was a three-case smoke test: two cases passed, semantic average for completed judgments was 4.9, but the overall gate failed because of deterministic accuracy, p95 latency and one judge error. Automatic delivery also failed because staging SSH variables and secrets are empty. Therefore I describe the project as production-oriented and instrumented, not fully production-validated.
Likely follow-up	Follow-up: This answer demonstrates honesty and an evidence-based release standard.

50

If the quality gate fails, what do you do?

Strong answer	I inspect the failed report and corresponding trace, identify the earliest wrong stage, and decide whether the cause is the golden expectation, ingestion, retrieval, compression, grading, prompt/model or infrastructure. I fix locally, add or update tests, push a new commit, redeploy that new commit to staging and rerun evaluation. I do not waive the result by rerunning the same failed deployment until it happens to pass.
Likely follow-up	Follow-up: If the golden case is wrong, a human corrects it through review rather than changing it merely to make the build green.

F. Ownership, trade-offs and behavioral questions

Use these answers as structures, then replace generic details with your own exact experience and wording.

51 What was the hardest engineering problem?

Strong answer	A strong example is preserving source completeness while keeping prompts bounded. Small chunks retrieve well but can lose conditions, steps and table headers; full pages retain context but increase noise and cost. I separated immutable EvidenceUnits from smaller RetrievalSegments, added neighbor expansion, resolved candidates back to parent units and created deterministic compressed views. I then added tests for procedures, tables, retrieval IDs and prompt budgets. This demonstrates that I fixed the data representation instead of only tuning a prompt.
Likely follow-up	Follow-up: Present it in STAR order: situation, task, action, measurable/observable result.

52 Tell me about a design decision you would reconsider.

Strong answer	The deterministic compressor is a deliberate first choice because it is cheap and auditable, but lexical selection can miss semantic relevance. I would not replace it immediately. I would create a human-labeled compression benchmark, compare deterministic selection with an LLM extractor on recall, faithfulness, tokens and p95, and adopt a hybrid only if the improvement is significant. This shows willingness to revise a design through evidence rather than fashion.
Likely follow-up	Follow-up: Another valid answer is calibrating role-specific models after measuring trace-level cost and quality.

53 Describe a result that did not pass.

Strong answer	The latest stored three-case live smoke run failed the gate: two cases passed, p95 was around 130.5 seconds and one semantic judge call errored. I treated that as useful evidence rather than hiding it. It showed that the full staging benchmark and latency optimization were not complete, and it prevented me from claiming a production-ready release. My next step is to configure staging, trace the slow nodes, fix routing/calibration issues and rerun all 50 cases against an accepted baseline.
Likely follow-up	Follow-up: Hiring managers value the corrective loop more than a claim that every experiment passed.

54 How would you respond to a production failure?

Strong answer	I would first protect users: verify impact, roll back or disable the affected path if needed, and preserve trace IDs. Then I would inspect LangSmith and service logs without copying sensitive manual content into an issue. A human would validate the expected behavior from the manual, create a sanitized golden case, implement the earliest-stage fix and run the PR and release gates. After release I would monitor recurrence and document the root cause.
Likely follow-up	Follow-up: Separate incident mitigation from permanent regression prevention.

55 What would you improve with one more week?

Strong answer	First I would configure an isolated staging deployment and run the complete 50-case live evaluation. Second I would inspect the slowest traces and reduce p95 through prompt budgets, safe caching, concurrency tuning and model selection. Third I would add expected evidence IDs/pages and human semantic labels to strengthen retrieval and judge calibration. Finally I would enable branch protection only after the live gate is stable and reproducible.
Likely follow-up	Follow-up: Prioritize validation and reliability before adding more agent features.

56

What would you change for a regulated enterprise deployment?

Strong answer

I would add SSO, role-based access to document collections, encryption and secret-manager integration, explicit data retention/deletion, audit events and tenant isolation. LangSmith tracing would be reviewed for data residency and redaction or replaced with an approved self-hosted option. Deployment would use protected environments, signed images, vulnerability scanning and rollback. Evaluation sets would be versioned with approvals and model/prompt changes would have documented release evidence.

Likely follow-up

Follow-up: Also discuss manual licensing and whether content may be sent to an external model provider.

57

How do you know the judge is reliable?

Strong answer

I do not assume it is reliable merely because it is an LLM. I would create a set scored independently by domain experts, compare judge pass/fail and dimension scores with human labels, examine disagreement by category and tune the rubric using a held-out set. I would track judge model/version, temperature, prompt and error rate. Deterministic citation and status checks remain independent so one judge cannot approve everything.

Likely follow-up

Follow-up: Mention inter-rater agreement and periodic recalibration when models or manuals change.

58

Why should this project matter on your resume?

Strong answer

It demonstrates the complete lifecycle rather than only a chat interface: document modeling, hybrid retrieval, stateful orchestration, asynchronous APIs, distributed coordination, tracing, evaluation, CI gates and deployment controls. It also includes explicit limitations and measurable release criteria. The strongest signal is that failures become inspectable reports and human-approved regression cases instead of ad hoc prompt changes.

Likely follow-up

Follow-up: Use one concrete technical detail from each layer rather than listing every library.

59

What did you learn from this project?

Strong answer

RAG quality is primarily a systems problem. Chunk structure, evidence identity, retrieval fusion, context completeness, state boundaries and observability matter as much as the final model. I also learned that an evaluation framework is only credible when the dataset is human-reviewed, thresholds are calibrated and failed runs are reported honestly. Production readiness is a sequence of verified controls, not a label added after containerization.

Likely follow-up

Follow-up: Connect the lesson to how you would approach another domain such as legal, support or engineering documentation.

60

Give me your closing statement.

Strong answer

I designed this project so every answer has a traceable path from source manual to retrieval result, evidence decision, generated response and evaluation outcome. The system is not just a RAG demo; it has explicit state, bounded failure behavior, observability and release-gate code. I am also clear about what remains: the staging secrets, full live 50-case validation, accepted baseline and latency tuning. That combination of implementation depth and honest verification is how I would approach production AI work on a team.

Likely follow-up

Follow-up: Stop after this answer. A concise close is stronger than adding another tool list.

Last-minute rapid revision

Architecture in one breath

Say this

PDFs are ingested offline into EvidenceUnits and retrieval indexes. Streamlit calls FastAPI, Redis coordinates the request, LangGraph loads PostgreSQL memory, Chroma and BM25 retrieve, RRF fuses, context expands, the cross-encoder reranks, the grader checks coverage, the answer model cites evidence, the verifier corrects it, and LangSmith traces the run.

Evaluation in one breath

Say this

Fifty versioned golden cases run through the real API. Deterministic checks measure terms, manual routing, evidence status, citations, diagrams, errors and latency; a separate judge scores five semantic dimensions; fixed thresholds and an accepted baseline decide whether CI passes.

Current status in one breath

Say this

The production-oriented code is merged to main and the evaluation contract passed. The latest live artifact is only a three-case failed smoke run, and automatic delivery is blocked by missing staging SSH configuration, so the full live 50-case release validation remains the next milestone.

Numbers worth remembering

Fact	Value
Golden cases	50 across 10 categories
PR live subset	10 representative cases
Typical direct-question LLM calls	3
Retrieval retry	Maximum 1
Backend replicas	2
RRF constant	60
Minimum overall/term pass	85%
Minimum manual/status accuracy	90%
Minimum citation accuracy	95%
Semantic threshold	4.0/5 average and 85% pass
Maximum p95 evaluation latency	75 seconds
Allowed API/judge errors	0
Maximum accepted-baseline drop	3 percentage points

Interview-day checklist

- Carry a local or hosted demo path, but prepare screenshots in case network access fails.
- Open the repository, README architecture, LangSmith project and evaluation report before the call.
- Answer the question first, then explain one design reason and one trade-off.
- Never claim that the full release gate or latest production deployment passed when it did not.
- Use the exact project vocabulary: EvidenceUnit, weighted RRF, checkpoint, deterministic evaluator and semantic judge.
- Keep the one-minute overview under 75 seconds and leave room for follow-up questions.

Questions you should ask the interviewer

- How does your team define and approve golden evaluation cases?
- Which production traces and user feedback signals drive your AI release decisions?
- How do you isolate sensitive retrieval context when using hosted observability or model providers?
- What is the team's process for model, prompt and retrieval regression testing?
- Which reliability or latency constraint is hardest in your current AI systems?

Final reminder

You do not need to know every library method. Show that you understand the data flow, failure modes, evidence, measurements and trade-offs - and that you can separate what was built from what still needs validation.